

리액트로 일정관리 프로그래밍 프로젝트 가이드

📅 1~2주차는 프론트 완성 + Firestore 연동

☁️ 3~4주차는 백엔드/DB 연동

🎯 1.프로젝트 목표

최종 결과물(필수)

- 달력 화면에 일정 표시(조회)
- 일정 등록(제목/시작/끝/메모/색상 등)
- 일정 수정
- 일정 삭제
- 일정 데이터 저장(Firestore사용)

완성 기준(눈으로 확인 가능한 체크리스트)

- 달력에서 날짜 클릭 → “일정 등록 모달” 뜸
- 저장하면 달력에 바로 표시됨
- 일정 클릭 → “상세/수정/삭제” 가능
- 새로고침해도 데이터 유지됨(Firestore사용)

📚 2. 기술 스택

1~2주차(프론트만)

- React (Vite)
- FullCalendar (React용)
- 상태관리: useState/useEffect -> Redux Toolkit로 변경해 보기
- 저장: Firestore (JSON)

3~4주차(확장: 백엔드/DB)

- Node.js + Express (REST API)
- DB연동 라이브러리: firebase 또는 mysql

📅 3. 1개월 로드맵(주차별 목표)

Week 1 — 환경 세팅 + 달력 화면 + 더미 일정 표시

🎯 목표: FullCalendar를 화면에 띄우고, 일정 배열을 달력에 표시한다.

- Day 1: 프로젝트 생성, 폴더 구조 이해, 실행
 - Day 2: FullCalendar 설치/기본 렌더링
 - Day 3: events(배열)로 일정 표시
 - Day 4: 일정 데이터 모델(필드 설계)
 - Day 5: UI(헤더/사이드바/버튼/간단 CSS)
- 📌 Week1 산출물: “일정이 보이는 달력 화면” 스크린샷

Week 2 — CRUD(등록/수정/삭제) + Firestore 저장

🔗 목표: 클릭/드래그/모달로 CRUD 구현 + 새로고침 유지

- Day 1: 날짜 클릭(dateClick) → 등록 모달
 - Day 2: 일정 저장 → state에 반영 → 달력 갱신
 - Day 3: 일정 클릭(eventClick) → 상세/수정 모달
 - Day 4: 삭제 기능 + 확인(confirm)
 - Day 5: Firestore 저장/불러오기(useEffect)
- 🔗 Week2 산출물: "등록/수정/삭제 시연 영상(30초)" 또는 GIF

Week 3 — 백엔드 API(선택/권장) + 프론트 연동

🔗 목표: REST API로 CRUD 수행

- Day 1: Express 서버 생성, 테스트 라우트
 - Day 2: 일정 API 설계(GET/POST/PUT/DELETE)
 - Day 3: 임시 메모리 저장으로 동작 확인
 - Day 4: 프론트에서 axios/fetch로 연동
 - Day 5: 에러 처리, 로딩/토스트, 예외 케이스
- 🔗 Week3 산출물: Postman로 API 동작 스샷

Week 4 — DB 저장 + 품질(검증/정리/발표 준비)

🔗 목표: DB 연동 + 발표 가능한 완성도(문서/테스트/배포)

- Day 1~2: DB 테이블 설계 + 연결
 - Day 3: 필터(월/주/일 보기, 검색, 카테고리)
 - Day 4: README, 트러블슈팅 정리
 - Day 5: 발표자료(기능 시연 + 구조도 + 회고)
- 🔗 Week4 산출물: GitHub README + 발표 PPT(또는 md)

🔗 4. 프로젝트 실행 방법

4-1 프로젝트 생성(React + Vite)

```
npm create vite@latest calendar-app  
cd calendar-app  
npm i  
npm run dev
```

```
D:\083KOSM020251029\77Calendar-Project  
$ npm create vite@latest vite-step1  
  
> npx  
> create-vite vite-step1  
  
Select a framework:  
Vanilla  
  
Select a variant:  
o TypeScript  
● JavaScript
```

```
D:\083KOSM020251029\77Calendar-Project  
$ npm create vite@latest vite-step1  
  
> npx  
> create-vite vite-step1  
  
|  
◇ Select a framework:  
Vanilla  
  
◇ Select a variant:  
JavaScript  
  
◆ Use rolldown-vite (Experimental)?:  
o Yes  
● No
```

```
D:\083KOSM020251029\77Calendar-Project
$ npm create vite@latest vite-step1

> npx
> create-vite vite-step1

|
| ◊ Select a framework:
|   Vanilla
|
| ◊ Select a variant:
|   JavaScript
|
| ◊ Use rolldown-vite (Experimental)? :
|   No
|
| ◆ Install with npm and start now?
|   ● Yes / ○ No
```

4-2 FullCalendar 설치

```
npm i @fullcalendar/react @fullcalendar/daygrid @fullcalendar/timegrid
```

4-3 달력 띄우기(최소 코드)

```
// src/pages/CalendarPage.jsx
import FullCalendar from "@fullcalendar/react";
import dayGridPlugin from "@fullcalendar/daygrid";
import timeGridPlugin from "@fullcalendar/timegrid";
import interactionPlugin from "@fullcalendar/interaction";

export default function CalendarPage({ events, onClick, onEventClick }) {
  return (
    <FullCalendar
      plugins={[dayGridPlugin, timeGridPlugin, interactionPlugin]}
      initialView="dayGridMonth"
      headerToolbar={{
```

```

        left: "prev,next today",
        center: "title",
        right: "dayGridMonth,timeGridWeek,timeGridDay",
    })
    events={events}
    dateClick={onDateClick}
    eventClick={onEventClick}
    selectable={true}
  />
);
}

```

5 데이터 설계(제일 먼저 결정해야 하는 것)

일정은 "객체 1개 = 달력 1개 이벤트"입니다.

권장 일정 모델

```

{
  id: "uuid-문자열",      // 반드시 유니크
  title: "회의",
  start: "2025-12-24T10:00:00",
  end: "2025-12-24T11:00:00",
  memo: "zoom 링크...",
  color: "#3b82f6"       // 선택
}

```

왜 이렇게?

FullCalendar가 title/start/end를 기본으로 씁니다.

id가 있어야 "수정/삭제 대상"을 정확히 찾습니다.

6 CRUD 구현 순서

Step A) "등록"부터(가장 쉽고 성취감 큼)

1. 날짜 클릭 → 선택한 날짜를 state로 저장
2. 모달 열기
3. 저장 버튼 클릭 → events 배열에 push(정확히는 새 배열로 set)

Step B) "조회"는 자동으로 되는 구조 만들기

- 달력은 events={events}만 제대로 주면 자동 렌더링

Step C) "수정"

1. 이벤트 클릭 → 해당 event의 id를 저장
2. 기존 데이터로 모달 입력칸 채우기
3. 저장 시 events.map()으로 해당 id만 교체

Step D) "삭제"

- events.filter(e => e.id !== targetId)로 제거

7 Firestore 적용

저장/불러오기 패턴

```

import { useEffect, useState } from "react";
import {
  getFirestore,
  collection,
  addDoc,
  doc,
  updateDoc,
  deleteDoc,
  onSnapshot,
  query,
  orderBy,
  serverTimestamp,
} from "firebase/firestore";
import app from "../service/firebase";

const db = getFirestore(app);
const COLLECTION = "schedules"; // 📍 localStorage KEY 대신 Firestore 컬렉션 이름

const [events, setEvents] = useState([]);

// ☑ 1) Firestore에서 불러오기(실시간 구독)
useEffect(() => {
  const q = query(collection(db, COLLECTION), orderBy("SCHEDULE_START", "asc"));

  const unsub = onSnapshot(q, (snap) => {
    const rows = snap.docs.map((d) => {
      const data = d.data();
      return {
        id: d.id, // Firestore 문서 id
        title: data.SCHEDULE_TITLE,
        start: data.SCHEDULE_START,
        end: data.SCHEDULE_END,
        category: data.SCHEDULE_STATUS,
        content: data.SCHEDULE_CONTENT,
        color: data.COLOR,
      };
    });

    setEvents(rows); // ☑ localStorage setEvents(JSON.parse(saved)) 역할
  });

  return () => unsub(); // ☑ 구독 해제
}, []);

// ☑ 2) Firestore에 저장(생성)
const createEvent = async (event) => {
  // localStorage의 setItem 역할이 아니라 "DB에 INSERT" 개념
  await addDoc(collection(db, COLLECTION), {
    SCHEDULE_TITLE: event.title,
    SCHEDULE_START: event.start, // ISO 문자열 추천
    SCHEDULE_END: event.end,
  });

```

```

    SCHEDULE_STATUS: event.category || "",
    SCHEDULE_CONTENT: event.content || "",
    COLOR: event.color || "#2563eb",
    createdAt: serverTimestamp(),
    updatedAt: serverTimestamp(),
  });
};

// ☑ 3) Firestore 수정(UPDATE)
const updateEvent = async (id, patch) => {
  await updateDoc(doc(db, COLLECTION, id), {
    ...(patch.title !== undefined && { SCHEDULE_TITLE: patch.title }),
    ...(patch.start !== undefined && { SCHEDULE_START: patch.start }),
    ...(patch.end !== undefined && { SCHEDULE_END: patch.end }),
    ...(patch.category !== undefined && { SCHEDULE_STATUS: patch.category }),
    ...(patch.content !== undefined && { SCHEDULE_CONTENT: patch.content }),
    ...(patch.color !== undefined && { COLOR: patch.color }),
    updatedAt: serverTimestamp(),
  });
};

// ☑ 4) Firestore 삭제(DELETE)
const deleteEventById = async (id) => {
  await deleteDoc(doc(db, COLLECTION, id));
};

```

☑ 실수 방지 팁 (Firestore 버전)

Firestore는 문자열만 저장하는 게 아니라 객체 그대로 저장 가능 → JSON.stringify/parse 필요 없음
그래도 날짜는 ISO 문자열로 저장 추천

예: "2026-01-03T10:00:00"

FullCalendar가 가장 안정적으로 처리

onSnapshot을 쓰면 DB가 바뀔 때마다 자동 반영되므로

☑ localStorage의 "events 바뀔 때마다 setItem" 같은 저장 effect가 필요 없습니다.

추가/수정/삭제는 setEvents()로 직접 조작하지 말고

☑ addDoc/updateDoc/deleteDoc만 호출하면 onSnapshot이 자동으로 최신 이벤트를 다시 내려줍니다.

8 UI/UX 최소 요구사항

모달: 제목/시작/끝/메모 입력

삭제 시 confirm("정말 삭제?")

입력 검증:

- 제목 비면 저장 불가
 - end가 start보다 빠르면 저장 불가
- 피드백:
- 저장 완료/수정 완료/삭제 완료 알림(간단히 alert부터 시작 OK)

9 자기주도 학습 방법("무엇을, 어떤 순서로 공부?")

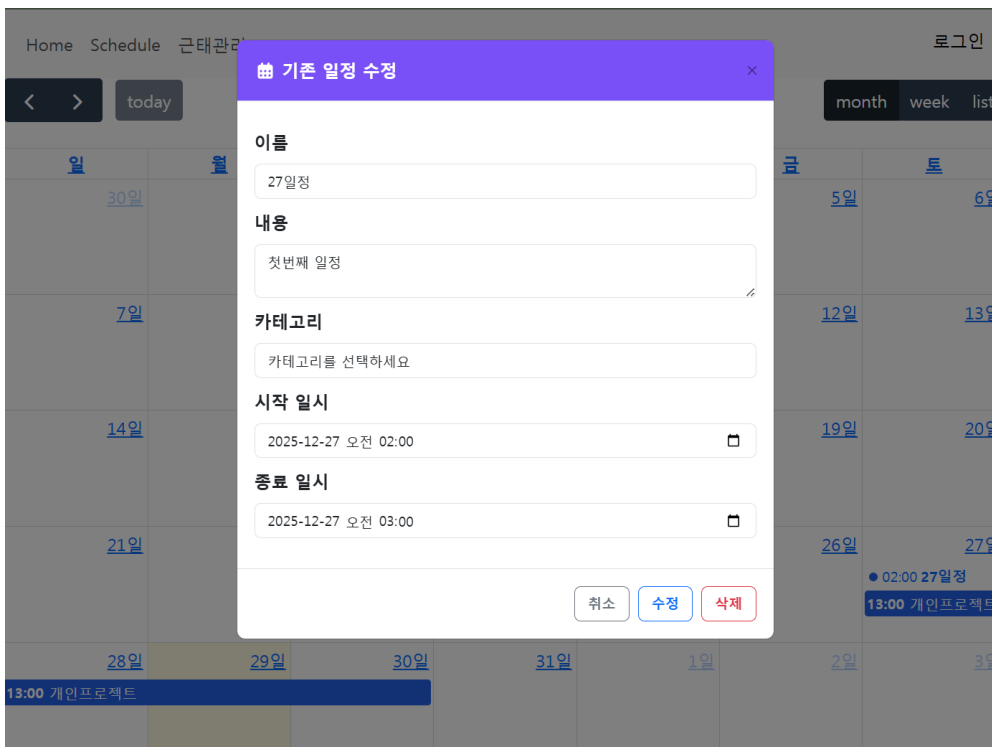
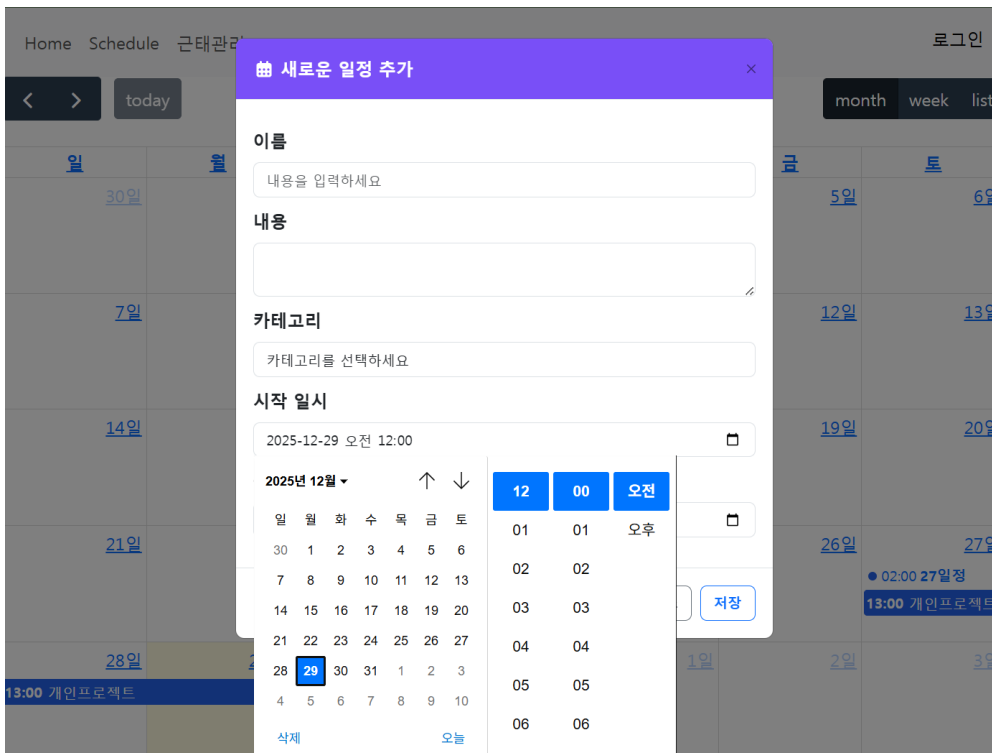
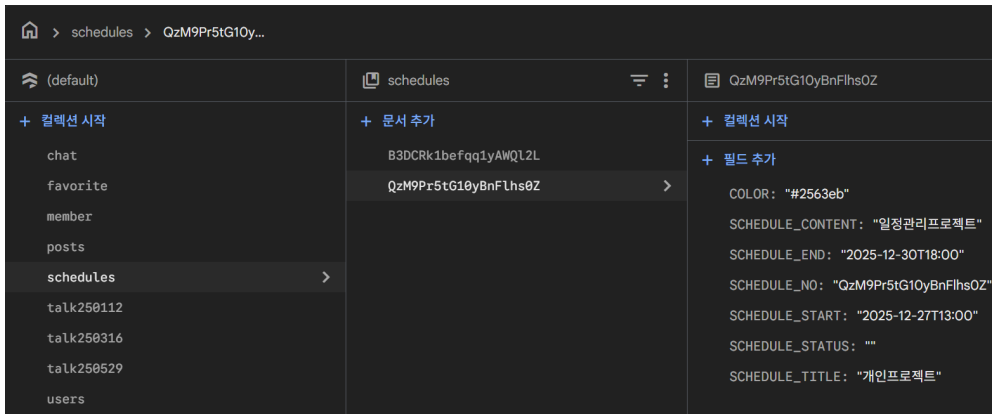
1.기능 1개 목표 세우기(20분)

- 예: "오늘은 dateClick으로 모달 띄우기"
- 2.공식 문서/예제 1개 읽기(30분)
- 키워드: FullCalendar React dateClick eventClick events
- 3.코딩 + 기록(90분)
- 성공/실패/해결"을 노트에 3줄로 남기기

10 트러블슈팅(초보자들이 자주 막히는 곳)

달력에 일정이 안 뜸

- events 배열 형태가 FullCalendar 요구와 다른 경우
- title/start/end가 있는지 확인
id가 없어서 수정/삭제가 꼬임
- 반드시 id를 만들 것(간단히 crypto.randomUUID() 가능)
날짜가 이상하게 표시됨
- start/end를 ISO 문자열로 통일
- "2025-12-24T10:00:00" 형태 권장
state 변경했는데 화면이 갱신 안 됨
- 배열을 직접 push하면 안 되고 setEvents([...events, newEvent])



HomeSchedule근태관리로그인

<>

today

2025년 12월

monthweeklist

일	월	화	수	목	금	토
30일	1일	2일	3일	4일	5일	6일
7일	8일	9일	10일	11일	12일	13일
14일	15일	16일	17일	18일	19일	20일
21일	22일	23일	24일	25일	26일	27일
					● 02:00 27일정 13:00 개인프로젝트	
28일	29일	30일	31일	1일	2일	3일
13:00 개인프로젝트						

HomeSchedule근태관리로그인

<>

today

2025년 12월 28일 – 2026년 1월 3일

monthweeklist

	12. 28. (일)	12. 29. (월)	12. 30. (화)	12. 31. (수)	1. 1. (목)	1. 2. (금)	1. 3. (토)
all-day							
오전 12시	개인프로젝트	개인프로젝트	00:00 - 18:00 개인프로젝트				
오전 1시							
오전 2시							
오전 3시							
오전 4시							
오전 5시							
오전 6시							
오전 7시							
오전 8시							
오전 9시							
오전 10시							

HomeSchedule근태관리로그인

<>

today

2025년 12월 28일 – 2026년 1월 3일

monthweeklist

일요일	2025년 12월 28일
all-day ● 개인프로젝트	
월요일	2025년 12월 29일
all-day ● 개인프로젝트	
화요일	2025년 12월 30일
00:00 - 18:00 ● 개인프로젝트	